Solidity

Solidity è un linguaggio di programmazione orientato agli oggetti, sviluppato appositamente per scrivere smart contract sulla blockchain di Ethereum. Viene spesso paragonato a Javascript, ma la sua sintassi assomiglia maggiormente al C. È un linguaggio fortemente tipizzato che possiede funzionalità minimali, in quanto enfatizza la limitazione dell'uso della memora e della CPU. Supporta nativamente l'ereditarietà, le librerie e l'uso di tipi di dato complessi

Ambiente di sviluppo

Solidity può essere utilizzato interfacciandosi con vari ambienti di sviluppo integrato (IDE), tra cui i più noti sono Remix, Truffle Suite e Visual Studio Code. In questi ambienti, gli strumenti dedicati al testing e al debugging sono essenziali per riuscire a garantire la sicurezza e l'affidabilità degli smart contract sviluppati.

Remix

Remix è un IDE fruibile direttamente su browser per lo sviluppo di smart contract in Solidity. La sua particolarità è che integra al suo interno una blockchain virtuale (Ethereum JavaScript VM), la quale permette allo sviluppatore di testare e distribuire (deploy) gli smart contract direttamente dall'IDE, fornendo anche strumenti di logging per tracciare e analizzare le transazioni.

EthFiddle

EthFiddle è uno strumento online per scrivere, condividere e testare frammenti di codice in Solidity, concettualmente molto simile a JSFiddle. Offre la comodità di eseguire e testare il codice direttamente all'interno del browser senza dover configurare alcun ambiente di sviluppo locale, risultando molto utile per l'apprendimento e la sperimentazione.

Primo contratto

Un esempio base in solidity è il contratto SimpleCounter, che mostra come mantenere e manipolare lo stato. Utilizza una variabile privata (_count) per tenere traccia del conteggio, un costruttore che inizializza questa variabile a 10 al momento del deployment, una funzione pubblica increment() che la aumenta di 1 e una funzione di sola lettura getCount() che ne restituisce il valore.

```
pragma solidity ^0.8.0; //Specifica la versione del compilatore
contract SimpleCounter //Dichiarazione del contratto, simile alla definizi
one di una classe in altri linguaggi
{
        uint private _count; //Variabile memorizzata sulla blockchain

        constructor() //Viene chiamato quando il contratto viene deployato
        {
                _count = 10;
        }

        function increment() public
        {
                _count += 1;
```

```
        }

        function getCount() public view returns (uint)
        {
                return _count;
        }
}
```

Testing

Per effettuare il testing su questo contratto in Remix, si procede con il deploy, si verifica che getCount() restituisce inizialmente 10, si invoca increment(), e infine si richiama nuovamente getCount() per assicurarsi che il valore sia salito a 11, ripetendo l'operazione per confermare il funzionamento continuo.

Deployment

Il deployment è la fase in cui, dopo aver compilato il codice sorgente in bytecode (usando strumenti come Truffle o script), il bytecode viene materialmente distribuito sulla blockchain. Questo avviene inviando una transazione all'indirizzo nullo (0x0...) contenente il bytecode come payload. Per compiere questa operazione è necessario pagare una quantità di gas, e al termine del processo al contratto viene assegnato un indirizzo unico sulla blockchain, essenziale per potervi interagire in futuro.

ABI

L'Application Binary Interface è l'interfaccia di comunicazione tra due programmi binari (l'equivalente di un'API per il bytecode), ovvero tra lo smart contract e chi lo chiama (che sia un'applicazione client o un altro contratto). Viene generata automaticamente durante la compilazione e contiene la firma di ciascuna funzione pubblica ed esterna (nome, tipi in ingresso e tipi di ritorno), escludendo le funzioni private e interne. Le applicazioni client usano l'ABI per costruire le transazioni corrette e per interpretare gli eventi emessi.

Bytecode

Il bytecode è il risultato della compilazione e comprende diverse sezioni strutturali:

- object: è il bytecode effettivo che viene eseguito dalla Ethereum Virtual Machine (EVM) e salvato sulla blockchain.
- opcodes: la rappresentazione leggibile delle operazioni specifiche eseguite dall'EVM.
- generatedSources: codice supplementare creato dal compilatore (es. wrapper o ottimizzazioni).
- linkReferences: indica dove collegare eventuali librerie esterne non ancora risolte in fase di compilazione.
- sourceMap: una mappa che collega il bytecode ai frammenti di codice sorgente originario, fondamentale per il debugging.

Simple Ponzi

Il contratto SimplePonzi implementa un basilare schema Ponzi in cui ogni nuovo partecipante ripaga il partecipante precedente. Tramite una funzione receive() esterna e "payable", il contratto impone che il nuovo investimento sia almeno il 10% più grande dell'investimento corrente. Se la condizione

è rispettata, il contratto aggiorna l'investitore e l'ammontare corrente, e invia immediatamente i fondi al precedente investitore.

```
pragma solidity ^0.8.0; //Specifica la versione del compilatore
contract SimplePonzi
{
        address payable _currentInvestor;
        uint256 public _currentInvestment = 0; //non occorre inizializzare
le variabili a zero, è uno spreco di gas!

        receive() external payable public
        {
                uint256 minimumInvestment = _currentInvestment * 11/10; //
il nuovo investimento deve essere > 10% di quello corrente
                require(msg.value > minimumInvestment);
                address payable previousInvestor = _currentInvestor;
                _currentInvestor = payable(msg.sender);
                _currentInvestment = msg.value;
                previousInvestor.transfer(msg.value);
        }
}
```

Testing di Simple Ponzi

Le fasi di testing per questo contratto prevedono:

→ verificare che l'investitore e l'investimento iniziale siano nulli.

→ fare un investimento e verificare il corretto aggiornamento delle variabili.

→ provare ad inviare un importo inferiore alla soglia (+10%) e assicurarsi che la transazione fallisca.

→ effettuare un secondo investimento valido e verificare che il primo investitore riceva i fondi.

Variabili

In Solidity le variabili si dividono in base a dove risiedono in memoria:

- **Variabili di storage**: hanno un costo elevato perché vengono memorizzate permanentemente sulla blockchain. Vengono dichiarate a livello di contratto (fuori dalle funzioni) e non necessitano di keyword specifiche, anche se il modificatore `storage` può essere usato nelle funzioni.
- **Variabili volatili**: sono economiche e non lasciano traccia sulla blockchain (esistono solo durante la chiamata della funzione). Si dividono in:
  - **memory**: dichiarate a livello di funzione, usate per tipi di dati complessi come array e struct.
  - **calldata**: utilizzate per i parametri delle funzioni esterne. Sono immutevoli e più efficienti in termini di gas rispetto alla memory.

Terzo contratto

```
contract MemoryStorageCalldata
{
        uint[5] public numbers = [1, 2, 3, 4, 5];

        function memoryWorking() public view returns (uint[5] memory)
        {
                uint[5] memory A = numbers;
                A[0] = 99;
                return numbers;
        }

        function storageWorking() public returns (uint[5] memory)
        {
                uint[5] storage B = numbers;
                B[0] = 99;
                return numbers;
        }

        function calldataWorking(uint[] calldata _input) external view ret
urns (uint[5] memory)
        {
                require(_input.length > 0, "wrong number of parameters");

                uint[5] memory A = numbers;
                A[0] = _input[0];

                return numbers;
        }
}
```

Tipi primitivi

I tipi di dati base del linguaggio includono:

→ **booleani**: bool.

→ **interi**: con segno (int / int256) o senza segno (uint / uint256), con disponibilità di versioni ridotte come int8 o uint16.

→ **indirizzi**: address (indirizzo a 20 byte) e address payable (che può ricevere Ether).

→ **byte e stringhe**: byte di lunghezza fissa (bytes1 a bytes32), di lunghezza variabile (bytes), e stringhe testuali (string).

→ **enumerazioni**: enum.

→ Esistono anche tipi decimali (fixed, ufixed), ma non sono ancora pienamente supportati. Per evitare imprecisioni nei calcoli matematici, in Solidity le operazioni decimali si gestiscono usando interi scalati per un fattore fisso (ad esempio, 1.5 Ether diventa 1500000000000000000 wei).

Visibilità

Solidity utilizza i modificatori di visibilità per controllare gli accessi a variabili e metodi:

→ **private**: accessibili solo all'interno del contratto specifico in cui sono definite.

→ **internal**: accessibili nel contratto e anche nei contratti derivati (eredità).

→ **public**: accessibili sia dall'interno che dall'esterno. Per le variabili pubbliche Solidity crea un metodo getter in automatico.

→ **external**: accessibili solo dall'esterno, i loro dati vengono passati via calldata permettendo un forte risparmio di gas.

Ci sono anche modificatori per il comportamento verso lo stato:

→ **view**: promettono di non modificare lo stato (solo lettura), non costando gas in quanto non effettuano transazioni.

→ **pure**: non leggono e non modificano lo stato, usate per fare calcoli basati unicamente sui parametri passati.

→ **payable**: abilitano la funzione a ricevere Ether.

Design Pattern

Owner

Il pattern Owner consiste nel memorizzare l'indirizzo del deployer del contratto, considerandolo il proprietario legittimo. Questo indirizzo viene letto tramite `msg.sender` e salvato in una variabile di tipo address all'interno del costruttore, che viene eseguito automaticamente durante la distribuzione sulla blockchain. Verificando l'indirizzo di chi invoca i metodi, il contratto permette di eseguire operazioni critiche solo al proprietario, ponendo le basi per l'autenticazione.

Testing del contratto Ownable

Per effettuare il testing del contratto Ownable si eseguono i seguenti passaggi:

1. verificare che l'indirizzo restituito chiamando getOwner() coincida con quello di chi ha distribuito il contratto.
2. verificare che lo stato iniziale della variabile (tramite getState()) sia 0.
3. chiamare setState() con un nuovo valore usando l'indirizzo del proprietario, assicurandosi che lo stato si aggiorni.
4. **controllo del proprietario**: tentare di richiamare setState() con un indirizzo diverso da quello del proprietario e verificare che la transazione fallisca mostrando l'errore "Not owner!"

Modificatori

Un modificatore (modifier) in Solidity è una funzione speciale utilizzata per alterare il comportamento delle funzioni di uno smart contract, permettendo di inserire un controllo pre-condizione prima della loro esecuzione.

Owner con modificatore

Applicandolo al concetto di Owner, si definisce un modificatore chiamato onlyOwner() che contiene il controllo `require(msg.sender == _owner, "Not owner!");` seguito dal simbolo `_;` (che ordina di continuare l'esecuzione della funzione originaria).

Aggiungendo questo modificatore alla firma di una funzione, ci si assicura in modo pulito che quella funzione possa essere invocata esclusivamente dal proprietario del contratto.

Authorization

A differenza dell'Owner (che concentra i poteri in un unico account), il pattern Authorization distribuisce i privilegi su diversi livelli ad una pluralità di utenti. Utilizza una mappa di indirizzi (hash table) per salvare i "super utenti" e i loro rispettivi permessi. Questo approccio offre un controllo molto più granulare sulle azioni specifiche o sulle modifiche di stato, riducendo drasticamente il rischio di abuso di potere.

Testing del contratto Authorization

Il testing sul contratto Authorization prevede di:

1. verificare che il deployer possieda il livello di permesso massimo (ad esempio 3) subito dopo la distribuzione.
2. assegnare nuovi permessi (chiamando la funzione setPermission()) ad altri account e verificare il corretto aggiornamento.
3. accertarsi dell'accesso alle funzioni, provando ad eseguire metodi protetti con indirizzi dotati dei privilegi adeguati.
4. testare il controllo dei permessi di accesso, richiamando le funzioni con account sprovvisti del livello richiesto e assicurandosi che tali transazioni vengano respinte.

Contract Composer

Il pattern Contract Composer si fonda sulla suddivisione di uno smart contract grande e complesso in molteplici contratti più piccoli e specializzati. Assegnando ad ogni contratto una responsabilità definita e chiara, il sistema nel suo complesso risulta molto più modulare, flessibile e facile da gestire, favorendo la riusabilità dei componenti in altre sezioni o progetti.

Facade

Il pattern Facade prevede la creazione di un contratto principale che agisce da interfaccia unica per gestire l'interazione con i vari contratti sottostanti (componenti), nascondendone la complessità all'utente.

Esempio Facade

Supponiamo di dover sviluppare un sistema di voto decentralizzato.

VotingFacade è l'interfaccia pubblica unica. Sotto di esso vi sono:

→ **VoteRecorder**: aggiunge le soluzioni votabili e conta i voti.

→ **VoteTracker**: traccia l'indirizzo di chi vota e restituisce il numero totale dei votanti.

Questi componenti ereditano da contratti base come **VoteBaseComposed** (per impostare il facade) e **Ownable** (per la gestione del proprietario).

Testing Facade

Il testing della struttura Facade si divide per moduli:

- **Ownable** e **VoteBaseComposed**: accertarsi che il proprietario sia settato al deploy, che l'indirizzo del Facade sia configurabile e che solo quest'ultimo possa chiudere la votazione.

- **VoteRecorder**: verificare l'aggiunta di opzioni, l'incremento corretto dei voti e che non si possa votare se la sessione è chiusa o la soluzione non esiste.
- **VoteTracker**: testare che un utente possa registrarsi, che nessuno possa votare due volte e che il conteggio totale si aggiorni.
- **VotingFacade**: assicurarsi che le chiamate pubbliche vengano reindirizzate in modo corretto a Tracker e Recorder, e che la chiusura del voto sia esclusiva del proprietario.

Migration

Il pattern Migration (conosciuto anche come Proxy) serve a superare il limite dell'immutabilità degli smart contract, permettendone l'aggiornamento e la gestione nel tempo. Si avvale di un contratto proxy che contiene un puntatore all'indirizzo del contratto reale contenente la logica. Qualora sia necessario distribuire una versione aggiornata o corretta del software, è sufficiente modificare l'indirizzo memorizzato nel proxy per farlo puntare al nuovo contratto. Questo stratagemma permette di evolvere il sistema senza dover cambiare l'interfaccia pubblica o l'indirizzo a cui si interfacciano i client.

Truffle

- **Compilazione, Migrazioni** (deploy), **Test (**Web3.js.)
- `npm install -g truffle`
- `truffle init`: crea una nuova struttura di cartelle con tutti i file di configurazione necessari per iniziare a lavorare con Truffle.
- `truffle compile`: compila i contratti
- `truffle migrate`: esegue le migrazioni e distribuisce i contratti.
- `truffle test`: esegue i test scritti in javascript salvati nella cartella test/.

Ganache

- **Emulatore**: Simula il funzionamento di un nodo Ethereum in un ambiente locale e controllato.
- **Interfaccia grafica**: permette di visualizzare le transazioni e gli stati dei contratti.

Mocha

- npm install --save-dev mocha
- framework di test JavaScript
- rende i test asincroni semplici.
- mappa le eccezioni non catturate ai casi di test corretti.

Chai

- npm install --save-dev chai
- npm install --save-dev chai-as-promised
- framework di test JavaScript
- Behavior-Driven Development basati su catene.
  - Es: expect(x).to.be.a('string');
- Test-Driven Development più classica.
  - Es: assert.equal(y, 123);

Web3

Web3 è una libreria JavaScript (installabile tramite `npm install web3`) progettata per interagire con un nodo Ethereum, sia esso locale o remoto. Fornisce un'interfaccia che permette di inviare transazioni, interagire con gli smart contract e accedere ai dati registrati sulla blockchain. Per la comunicazione, Web3 può utilizzare diverse tipologie di connessione: HTTP, IPC (ideale per lo scambio di dati tra un'applicazione client e un nodo server) o WebSocket (che garantisce una comunicazione full-duplex continua e in tempo reale sfruttando una singola connessione TCP).

Testing con Truffle

In ambito di test con Truffle, la funzione `contract` ha il compito di creare suite di test separate per ogni singolo contratto, garantendo così un ambiente di esecuzione isolato e pulito. All'interno di queste suite, i casi di test specifici vengono definiti con la funzione `it`. Tra le pratiche comuni di testing vi è la verifica della corretta distribuzione dei contratti (assicurandosi che gli indirizzi generati non siano nulli). Ad esempio, testando un contratto Proxy, si utilizzano asserzioni come `.should.be.fulfilled` e `.should.be.rejected` per garantire che solo l'account proprietario (owner) possa aggiornare il puntatore al contratto logico. Infine, impiegando Web3.js per collegarsi all'emulatore locale (Ganache), si creano istanze per testare il corretto reindirizzamento delle funzioni attraverso il Proxy, sia prima che dopo un aggiornamento della logica.

Emergency Stop

L'Emergency Stop è un meccanismo di sicurezza fondamentale che consente di interrompere la maggior parte delle operazioni di un contratto nel caso in cui si verifichino anomalie o vengano scoperte vulnerabilità critiche. L'obiettivo è bloccare le transazioni per prevenire danni ulteriori. Tuttavia, un aspetto cruciale di questo pattern è che, nonostante il blocco generale, funzionalità essenziali come il prelievo dei fondi devono rimanere attive, in modo da consentire agli utenti di poter sempre recuperare i propri asset. Tecnologicamente si applica tramite modificatori (come `onlyIfUnlocked()`) che limitano l'esecuzione e funzioni esclusive per il proprietario per attivare o disattivare lo stato di blocco (`lock()` e `unlock()`).

Gradual Ponzi

Il contratto Gradual Ponzi è una variante di schema Ponzi dotata di sistema di blocco (eredita da Stoppable), in cui l'importo investito da un nuovo utente non va a un singolo partecipante, ma viene distribuito in parti uguali tra tutti gli investitori precedenti. Il proprietario del contratto figura come il primo investitore per trarre beneficio dal sistema. Gli utenti versano tramite la funzione `invest()` (che richiede una somma minima ed è bloccabile in emergenza) e ritirano i fondi accumulati tramite `withdraw()`.

Testing del contratto Gradual Ponzi

Il testing di questa struttura richiede di verificare molteplici scenari:

- **Soglia**: i versamenti inferiori alla quota minima non devono essere accettati.
- **Investimento**: un investimento valido deve aggiungere l'utente all'array degli investitori, aggiornare i bilanci ed emettere i relativi eventi.
- **Distribuzione**: assicurarsi matematicamente che l'ammontare versato venga diviso equamente.
- **Prelievo**: l'utente deve poter prelevare, e il suo saldo locale deve essere azzerato immediatamente dopo l'operazione.
- **Sicurezza**: solo l'owner deve poter bloccare/sbloccare il contratto.
- **Verifica lock**: durante lo stato di emergenza, l'azione di investimento deve fallire, mentre il prelievo deve rimanere consentito.

Durante la scrittura dei test per i contratti (come il Gradual Ponzi), due strumenti risultano essere essenziali:

→ **Web3.js**: viene impiegata direttamente nel codice di test per instaurare l'interazione con la blockchain, permettendo di richiamare le funzioni del contratto e di estrarre le informazioni sugli account.

→ **BigNumber**: poiché Ethereum gestisce valori enormi (come le frazioni di Ether in Wei che non sono supportate dai normali tipi numerici JavaScript), si utilizza la libreria `bignumber.js` (npm install bignumber.js) per gestire in sicurezza e senza perdita di precisione i numeri di grandi dimensioni. Questo è strettamente necessario per poter confrontare in maniera corretta i bilanci degli utenti e le cifre trasferite.

Eventi

Gli eventi in Solidity sono strumenti utilizzati per catturare informazioni e comunicarle al mondo esterno, risultando estremamente utili per l'interazione con server o interfacce utente (UI). A differenza delle normali variabili, gli eventi vengono salvati all'interno della cronologia delle transazioni della blockchain, offrendo un metodo molto più economico in termini di consumo di gas per memorizzare dati a lungo termine (poiché possono essere recuperati anche a transazione conclusa).

Esempio: `event nome(tipo parametro1, tipo parametro2, ..., tipo parametroN);`

Si dichiarano con la parola chiave `event`, e i loro parametri possono essere marcati come `indexed` per creare degli indici che permettono di filtrare e ricercare le informazioni in modo estremamente efficiente. L'emissione effettiva avviene chiamando la parola chiave `emit`:

`emit nome(parametro1, parametro2, ..., parametroN);`

In fase di testing, per verificare la corretta emissione di un evento, è necessario leggere i log restituiti dalla transazione (ad esempio, **transaction.logs**), iterando su di essi fino a trovare il nome dell'evento desiderato ed estraendone i valori per confrontarli con quelli attesi.

Costi

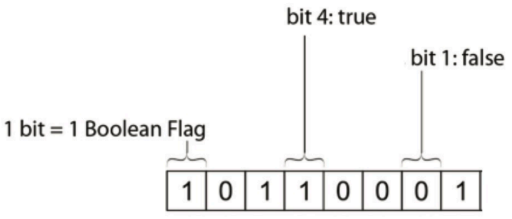
Best practices per ottimizzare il consumo di gas

Per ridurre i costi di esecuzione (gas) degli smart contract, è consigliato seguire diverse best practices:

- **Impacchettamento**: dichiarare le variabili di storage dello stesso tipo in modo consecutivo permette al compilatore di “impacchettarle” occupando meno slot di memoria.
- **Uso di uint256**: se non è possibile impacchettare i dati, è meglio usare uint256 al posto di interi più piccoli per evitare inutili costi di conversione.
- **Chiamate esterne**: limitarne l’uso preferendo singole funzioni multiuso.
- **Mapping vs array**: le tabelle hash (mapping) sono strutturalmente più efficienti rispetto ai classici array.
- **Array a dimensione fissa**: evitare l’uso di array dinamici previene costose operazioni di gestione automatica della memoria.
- **Valori predefiniti**: evitare di inizializzare esplicitamente le variabili a zero (es. uint256 x = 0), poiché Solidity lo fa già in automatico e forzarlo rappresenta uno spreco.
- **Short circuit**: ordinare in modo strategico le espressioni con operatori logici per ridurre la probabilità che il sistema debba valutare anche il secondo termine.
- **Stringhe brevi**: mantenere le stringhe costanti sotto la soglia dei 32 byte.
- **Scambio in linea singola**: per scambiare il valore di due variabili senza usare variabili temporanee di appoggio, utilizzare l’istruzione nativa `(a,b)=(b,a);`.
- **Valori hardcoded**: impostare direttamente valori già noti anziché farli calcolare al contratto, per evitare computazioni inutili.

BooleanBox

In Solidity, i valori booleani standard vengono memorizzati come interi senza segno di almeno un byte, poiché i dati devono essere indirizzabili. Il BooleanBox è un metodo per compattare le informazioni, ottenendo otto valori booleani da un singolo byte, anziché utilizzare otto variabili distinte. Questo porta ad una riduzione del costo del gas per la memorizzazione e, grazie alle operazioni bitwise (bit a bit), permette di leggere o modificare ogni singolo bit individualmente.



Le sue funzioni chiave includono:

- **set / get**: impostano o restituiscono lo stato di tutti gli 8 bit in un colpo solo.
- **setFlagTrue / setFlagFalse**: attivano (1) o disattivano (0) un singolo bit specificato tramite un indice.
- **setFlagTrueMask / setFlagFalseMask**: utilizzano una maschera binaria per attivare o disattivare più bit contemporaneamente.
- **getFlag**: restituisce lo stato attuale di uno specifico bit tramite indice.
- **toggleFlag**: inverte lo stato di uno specifico bit.

Sviluppando con questo metodo, bisogna fare molta attenzione a gestire correttamente gli indici per evitare errori o overflow.

Testing

Il testing per le logiche di manipolazione dei bit (come il BooleanBox) prevede la verifica rigorosa di queste operazioni:

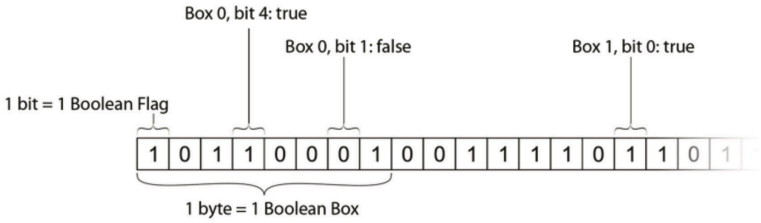
- **Impostazione di un bit specifico**: utilizzare setFlagTrue o setFlagFalse e verificare che solo il bit desiderato sia effettivamente cambiato, lasciando gli altri inalterati.
- **Impostazione con maschere**: utilizzare le funzioni Mask e verificare che i bit del BooleanBox corrispondente agli "1" della maschera applicata siano stati modificati correttamente.
- **Toggle**: assicurarsi che toggleFlag riesca a invertire lo stato di un bit in entrambe le direzioni (da 0 a 1 e da 1 a 0).
- **Letture**: usare la funzione get per verificare l'intero byte e getFlag per assicurarsi che la lettura del singolo bit rifletta il valore corretto e attuale.

BooleanBoxes

Il concetto di BooleanBoxes espande quello del BooleanBox singolo. Invece di usare un solo byte, utilizza un array di 32 byte (pari a 256 bit) per occupare esattamente un'intera word (parola) della Ethereum Virtual Machine (EVM). La sintassi per dichiararlo è:

```
uint8[32] myBooleanBoxArray;
```

Le funzionalità e i metodi rimangono identici a quelli del BooleanBox singolo, ma per utilizzarli è necessario fornire un indice aggiuntivo che specifichi su quale box (ovvero su quale degli array di byte) si intende operare.



Testnet

Sepolia

Sepolia è una delle reti di test di Ethereum, impiegata per testare gli smart contract prima del loro effettivo lancio sulla rete principale. Questa rete va a sostituire le reti di test precedenti, come ad esempio Ropsten. Su Sepolia non viene utilizzato Ether reale, ma una specifica tipologia di Ether fittizio dedicato esclusivamente ai test. Inoltre, questa rete supporta il meccanismo del Proof of Stake, aiutando gli sviluppatori a simulare e testare l'ambiente post-Merge.

MetaMask

MetaMask è un wallet software che funge da gateway per interagire con le applicazioni decentralizzate sulla blockchain di Ethereum. Si utilizza direttamente attraverso il browser web e offre la versatilità di poter interagire anche con diverse altre blockchain.

Faucet

I Faucet di Ethereum su Sepolia sono servizi che distribuiscono gratuitamente Ether di test agli sviluppatori, un elemento indispensabile per permettere l'esecuzione dei test sulle reti. Questi sistemi erogano una quantità limitata di Ether e sono spesso implementati e offerti da siti web che forniscono servizi legati allo sviluppo degli smart contract.

Deploy su Sepolia

Per effettuare il Deploy su Sepolia partendo da Remix, è necessario seguire una specifica procedura:

- assicurarsi innanzitutto che MetaMask sia impostato sulla rete Sepolia e che il proprio wallet sia connesso all'ambiente Remix.
- nella scheda denominata "Deploy & run transactions", bisogna selezionare "Injected Web3" come ambiente di esecuzione, in modo da poter utilizzare MetaMask.
- scegliere il contratto desiderato e cliccare su "Deploy".
- infine, confermare la transazione direttamente dalla finestra di MetaMask.